

# JARVIS Engineering System (JES)

**Versión:** 2026.0 (Draft)

**Estado:** En desarrollo

---

## 1. Introducción

El **JARVIS Engineering System (JES)** define el modelo mediante el cual se desarrolla el proyecto JARVIS.

Su propósito no es establecer un conjunto de herramientas ni imponer una metodología tradicional de desarrollo. Su objetivo es diseñar un sistema de ingeniería reproducible donde humanos e inteligencias artificiales colaboren bajo una arquitectura claramente definida.

El JES describe:

- los principios que gobiernan el desarrollo;
- los roles que intervienen en el proceso;
- el flujo de trabajo entre humanos y agentes;
- la organización del conocimiento del proyecto;
- las normas que garantizan la evolución sostenible del sistema.

El JES es independiente de cualquier herramienta concreta. Las tecnologías evolucionan; los principios de ingeniería permanecen.

---

## 2. Visión

JARVIS no pretende ser únicamente un proyecto software.

Su objetivo es demostrar que es posible construir sistemas complejos mediante una colaboración estructurada entre un ingeniero y agentes de inteligencia artificial.

El conocimiento debe permanecer dentro del proyecto.

Las decisiones deben ser reproducibles.

La arquitectura debe ser comprensible.

La implementación debe poder delegarse sin perder el control técnico.

---

### 3. Misión

Construir un sistema de ingeniería donde la inteligencia artificial incremente la productividad sin sustituir la responsabilidad técnica del ingeniero.

---

## 4. Filosofía

### 4.1 Ingeniería antes que programación

El objetivo del proyecto no es escribir código.

El objetivo es resolver problemas mediante ingeniería.

El código representa únicamente la implementación de una decisión de diseño.

Diseñar tiene prioridad sobre implementar.

Comprender tiene prioridad sobre producir.

Validar tiene prioridad sobre finalizar.

---

### 4.2 El conocimiento es el activo principal

El verdadero producto del proyecto no es el código fuente.

El verdadero producto es el conocimiento que permite comprender, mantener y evolucionar el sistema.

Todo conocimiento relevante debe permanecer dentro del repositorio.

Ninguna decisión importante debe depender de la memoria de una conversación o de una persona.

---

## 4.3 El repositorio es la fuente de verdad

El repositorio constituye la representación completa del sistema.

Debe contener:

- arquitectura;
- documentación;
- contratos;
- decisiones;
- reglas;
- implementación;
- pruebas.

Cualquier ingeniero debe poder comprender el sistema únicamente consultando el repositorio.

---

## 4.4 La inteligencia artificial es un colaborador técnico

Los agentes de IA forman parte del proceso de ingeniería.

No constituyen la autoridad técnica del proyecto.

Su responsabilidad consiste en asistir, acelerar e implementar decisiones previamente definidas.

Toda decisión arquitectónica permanece bajo responsabilidad humana.

---

## 4.5 Simplicidad como principio de diseño

Cada componente añadido incrementa la complejidad del sistema.

Toda nueva herramienta, dependencia o abstracción deberá justificar claramente el valor que aporta.

La complejidad accidental debe eliminarse de forma continua.

---

## 5. Principios Fundamentales

### P1 — Human Authority Principle

La autoridad técnica nunca se delega.

Las inteligencias artificiales pueden:

- proponer;
- implementar;
- analizar;
- revisar.

La decisión final pertenece siempre al ingeniero responsable.

---

### P2 — AI Implementation Principle

La implementación podrá delegarse.

El diseño no.

---

### P3 — Repository First

Todo conocimiento debe existir dentro del proyecto.

No se aceptan dependencias de conversaciones privadas, memoria personal o contexto externo.

---

### P4 — Documentation Driven Engineering

La documentación forma parte del sistema.

No constituye una tarea posterior.

---

## P5 — Deterministic Engineering

Todo cambio debe ser:

- reproducible;
  - verificable;
  - trazable.
- 

## P6 — Replaceable Tools

Las herramientas son intercambiables.

La arquitectura del sistema nunca dependerá de un proveedor específico.

---

## P7 — Context over Memory

Los agentes deben trabajar utilizando contexto explícito.

Nunca se asumirá que recuerdan conversaciones anteriores.

---

## P8 — Continuous Validation

Todo cambio debe finalizar con validación.

No existe implementación sin verificación.

---

# 6. Modelo Organizativo

El JES organiza el desarrollo mediante **roles**, no mediante herramientas.

Cada rol representa una responsabilidad de ingeniería.

Las herramientas únicamente implementan dichos roles.

Esta separación garantiza la independencia tecnológica del sistema.

---

## 7. Roles de Ingeniería

### Human System Architect

Responsabilidades:

- definir la visión del sistema;
- tomar decisiones arquitectónicas;
- aprobar cambios relevantes;
- validar el comportamiento final.

Autoridad: máxima.

---

### Chief Software Architect

Responsabilidades:

- analizar arquitectura;
- revisar diseños;
- proponer estrategias;
- evaluar decisiones técnicas;
- identificar riesgos.

No implementa funcionalidades rutinarias.

Su misión consiste en preservar la coherencia del sistema.

---

### Principal Engineer

Responsabilidades:

- analizar grandes modificaciones;
- diseñar refactorizaciones;
- estudiar impacto arquitectónico;
- resolver problemas complejos.

Actúa sobre cambios transversales.

---

## Software Engineer

Responsabilidades:

- implementar funcionalidades;
- modificar código;
- generar pruebas;
- mantener documentación técnica;
- aplicar cambios definidos.

Representa el rol operativo del sistema.

---

## Quality Engineer

Responsabilidades:

- verificar comportamiento;
  - ejecutar pruebas;
  - validar contratos;
  - comprobar regresiones;
  - asegurar calidad.
- 

## 8. Flujo Oficial de Desarrollo

Todo cambio deberá seguir el siguiente proceso:

1. Definición del problema.
2. Diseño de la solución.
3. Plan de implementación.
4. Implementación.
5. Ejecución de pruebas.
6. Revisión técnica.
7. Validación humana.
8. Integración.

Ninguna fase podrá omitirse salvo justificación explícita.

---

## 9. Ciclo de Ingeniería

Problema

↓

Análisis

↓

Diseño

↓

Plan

↓

Implementación

↓

Pruebas

↓

Revisión

↓

Validación

↓

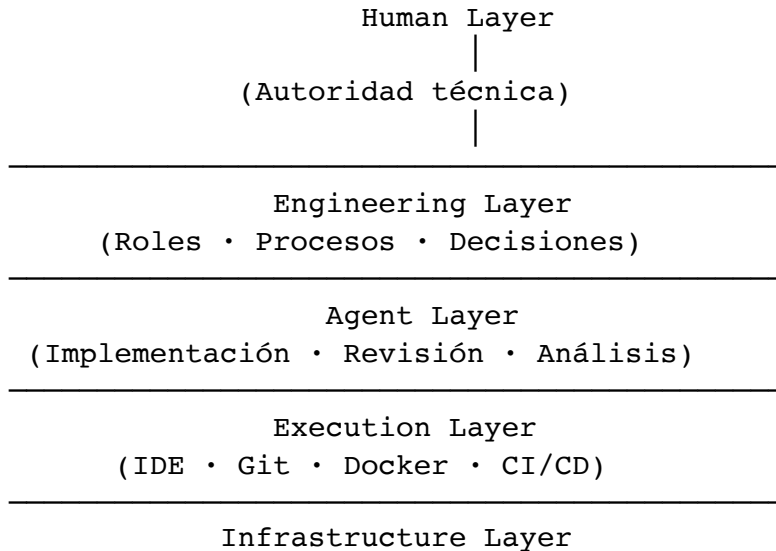
Integración

La implementación representa únicamente una fase dentro del ciclo completo.

---



## 10. Arquitectura Conceptual del JES



Las herramientas pertenecen exclusivamente a la capa de ejecución.

Nunca forman parte de la arquitectura conceptual.

---

## 11. Criterios de Incorporación de Herramientas

Una herramienta únicamente podrá incorporarse al JES si cumple, al menos, uno de los siguientes criterios:

- reduce tiempo de desarrollo;
- mejora la calidad del software;
- disminuye errores humanos;
- elimina complejidad existente;
- aumenta la reproducibilidad.

La popularidad o las tendencias del mercado no constituyen criterios válidos.

---

## 12. Definición de Éxito

El JES tendrá éxito cuando:

- un nuevo ingeniero pueda comprender el proyecto únicamente leyendo el repositorio;
  - cualquier agente pueda colaborar utilizando el contexto del proyecto;
  - las decisiones arquitectónicas permanezcan coherentes durante la evolución del sistema;
  - las herramientas puedan sustituirse sin modificar la metodología de trabajo;
  - el conocimiento permanezca dentro del proyecto y no dependa de conversaciones previas.
- 

## Estado del Documento

Este documento constituye la especificación fundacional del **JARVIS Engineering System**.

Las siguientes versiones desarrollarán:

- arquitectura documental;
- estructura del repositorio;
- configuración de agentes;
- reglas operativas;
- estándares de calidad;
- flujo Git;
- integración con herramientas;
- automatización del ciclo de ingeniería.